

TP 5 : Introduction à Bison & l'Analyse Syntaxique

Année universitaire : 2024 – 2025

Enseignante : Aida Lahouij

1. Objectif

L'objectif de cette séance de TP est d'introduire les étudiants à l'outil **Bison**, qui permet de générer automatiquement un analyseur syntaxique à partir d'un fichier de spécification. Ce fichier définit les règles de production de la grammaire ainsi que les actions sémantiques associées.

2. Utilisation de Bison

Bison est un générateur d'analyseurs pour les grammaires hors-contexte. Il est utilisé pour reconnaître les phrases d'un langage.

Bison génère du code source en C pour l'analyseur syntaxique à partir d'un fichier de spécification fourni (par exemple : monfichier.y). Le fichier généré aura le même nom que le fichier de spécification, mais avec l'extension .tab.c.

Par exemple, si votre fichier Bison s'appelle monfichier.y, le fichier C généré sera monfichier.tab.c.

Commande pour générer l'analyseur syntaxique :

```
bison monfichier.y
```

Ensuite, compilez le fichier source généré pour produire l'exécutable de l'analyseur :

```
gcc monfichier.tab.c -o monfichier.exe
```

Si gcc produit une erreur liée à la fonction manquante yywrap(), compilez en ajoutant l'option de liaison -ly.

Il est important de noter que Bison seul ne suffit pas pour construire un compilateur complet. Un analyseur lexical (ex. avec Flex), des actions sémantiques et des modules de génération de code doivent être ajoutés.

3. Format du fichier de spécification

Le format d' un fichier de spécifications est le suivant :

```
%{  
Déclarations en C : variables, constantes, fichiers d'en-tête, etc.  
%}  
  
Déclarations de jetons lexicaux  
  
Déclarations de précédence et de types  
  
%%  
  
Règles de production de la grammaire et actions associées :  
  
NonTerminal : liste_de_symboles { action C 1 }  
              | liste_de_symboles { action C 2 }  
              | ...  
              ;  
  
%%  
  
Routines C et fonction main
```

a) les symboles sont représentés par des identificateurs

- Terminaux : lexèmes provenant de l' analyse lexicale

- Non-terminaux : symboles internes à l' analyseur syntaxique

b) le code C est optionnel et est exécuté lorsque la règle est réduite

Pour exécuter l' analyseur syntaxique généré, il faut lui fournir comme entrée un flot d' unités lexicales. Ce flot est l' output obtenu après l' exécution de l' analyseur lexical.

4. Coordination entre les analyseurs lexical et syntaxique

Pour transmettre la sortie de l' analyseur lexical à l' analyseur syntaxique, une coordination entre les deux est nécessaire, comme illustré par la flèche rouge sur le schéma.

Étapes d' intégration :

- Définir les jetons dans l' analyseur syntaxique à l' aide de %token.
- Inclure le fichier d' en-tête généré par Bison dans le fichier de spécification Flex.

Dans le fichier Flex (monfichier.l), inclure dans la section des déclarations :

```
#include "monfichier.tab.h"

extern int yyval;
```

Pour que Bison génère un fichier .h en plus du code source en C, il faut utiliser l'option -d. La compilation s'effectue par conséquent comme suit :

```
>> bison -d monfichier.y

>> flex monfichier.l

>> gcc -o parser lex.yy.c monfichier.tab.c
```

Ou bien:

```
>> gcc -o parser lex.yy.c monfichier.tab.c main.c
```

dans le cas où la fonction main est dans un fichier séparé traduire en anglais

5. Tester Bison

5.1. Soit la spécification Flex suivante

```
/*fichier monfichier.l */
%{
/* fichier dans lequel est defini la macro constante NOMBRE */
```

```

#include "monfichier.tab.h"
#include <stdlib.h>

extern int yylval;
%}
%option noyywrap
%%
[0-9]+ {return NOMBRE;}
[ \t] ; /* ignore les blancs et tabulations */
\n return 0;
. return yytext[0];
%%

```

5.2. Soit la spécification Bison suivante

```

/*fichier monfichier.y */
%{
#include<stdio.h>
%}
%token NOMBRE /* liste des terminaux */
%%
expression: expression '+' terme
| expression '-' terme
| terme
;
terme: terme '*' facteur
| terme '/' facteur
| facteur
;
facteur: '(' expression ')'
| '-' facteur
| NOMBRE
;
%%
int yyerror(void)
{ fprintf(stderr, "erreur de syntaxe\n");
return 1;
}

```

5.3. Soit la fonction principale suivante

```
/*fichier monfichier.c */
```

```
int main(void)
{
  yyparse();
}
```

5.5. Exécuter les commandes suivantes

```
>> bison -d monfichier.y
```

La commande `bison -d monfichier.y` produit les fichiers :

- **monfichier.tab.c** qui contient le code en c de l'analyseur

et

- **monfichier.tab.h** qui contient la définition des codes des unités lexicales, afin qu'elles puissent être partagées par l'analyseur syntaxique et l'analyseur lexical.

```
>> flex monfichier.l
```

La commande `flex monfichier.l` produit le fichier `lex.yy.c` qui contient le code en c de l'analyseur lexical.

```
>> gcc -o analyseur lex.yy.c monfichier.tab.c monfichier.c
```

La commande `gcc` produit l'exécutable `analyseur` qui permet d'analyser des expressions arithmétiques.

5.6. Exemples d'exécution

- Analyse d'une expression correcte

```
>> analyseur
>> 1 + 2
```

C'est un exemple syntaxiquement correct donc on n'obtient rien à l'affichage

- Analyse d'une expression incorrecte

```
>> analyseur
>> 1 + A
```

erreur de syntaxe

5.7. Communication entre les analyseurs syntaxique et lexical :

- Le fichier lex.yy.c définit la fonction yylex() qui analyse le contenu du flux yyin jusqu' à détecter une unité lexicale. Elle renvoie alors le code de l' unité reconnue.
- L' analyseur syntaxique (yyparse()) appelle la fonction yylex() pour connaître la prochaine unité lexicale à traiter.
- Le fichier monfichier.tab.h associe un entier à tout symbole terminal.

```
# define YYTOKENTYPE
```

```
/* Put the tokens into the symbol table, so that GDB and
other debuggers know about them. */
```

```
enum yytokentype {
    NOMBRE = 258
};
```

5.8. Dans le fichier monfichier.y, ajouter les actions sémantiques suivantes aux règles de productions

```
%{
#include <stdio.h>
%}
%token NOMBRE
%%
calcul: expression {printf("%d\n", $1);}
;
expression: expression '+' terme {$$ = $1 + $3;}
| expression '-' terme {$$ = $1 - $3;}
| terme {$$ = $1;}
;
terme: terme '*' facteur {$$ = $1 * $3;}
| terme '/' facteur {$$ = $1 / $3;}
| facteur {$$ = $1;}
;
facteur: '(' expression ')' {$$ = $2;}
| '-' facteur {$$ = -$2;}
| NOMBRE {$$ = $1;}
```

```

;
%%
int yyerror(void)
{ fprintf(stderr, "erreur de syntaxe\n") ; return 1 ;}

```

5.9. Dans le fichier monfichier.l, associer une valeur à l'unité lexicale NOMBRE

```

%{
#include "monfichier.tab.h"
#include <stdlib.h>
extern int yylval ;
}%
%option noyywrap
%%
[0-9]+ {yylval=atoi(yytext);return NOMBRE;}
[ \t] ; /* ignore les blancs et tabulations */
\n return 0;
. return yytext[0];
%%

```

5.10. Ré-exécuter les commandes

```

>> bison -d monfichier.y
>> flex monfichier.l
>> gcc -o analyseur lex.yy.c monfichier.tab.c monfichier.c

```

5.11. Tester les exemples d'exécution suivants

Exemple 1

```

>> analyseur
>> 1 + 6*4

```

-> 25

Exemple 2

```

>> analyseur
>> 1 / 2

```

-> 0

Exercices:

Exercice 1.

Développer un analyseur syntaxique capable de reconnaître des mots valides (a*b*) suivis d'un saut de ligne.

Exercice2.

Développer un analyseur syntaxique capable de reconnaître des mots valides (a^*b^*) suivis d'un point virgule puis un saut de ligne.

Exercice3.

Développer un analyseur syntaxique capable de reconnaître les phrases:

le chat mange la souris

la souris attrape le chat